OFFICIAL SYSTEM ARCHITECTURE & MASTER MANUAL

AETHER SOVEREIGN LAYER-1 BLOCKCHAIN

Production Multi-Tenant Currency Engine, Dynamic Founder Gas Splitter, Runtime Transfer Restriction Hooks & End-to-End Deployment Guide

SYSTEM SPECIFICATIONS & KEY ATTRIBUTES

Framework Engine:	Cosmos SDK v0.52+ (Store v2 / Collections Architecture)
Consensus Protocol:	CometBFT Byzantine Fault Tolerant PoS (1-2s Finality)
Core Custom Modules:	x/currencies (Multi-Tenant Tokens), x/feeshare (Revenue Sharing)
Founder Gas Revenue:	2000 bps (20%) of ALL network fees routed dynamically to Founder
Token Economic Modes:	CAPPED (Supply headroom), UNCAPPED (Admin), COLLATERALIZED (Escrow)
Runtime Protection:	SendRestrictionFn (Up to 10% Creator Tax, Dynamic Pause Freeze)
Client Queries:	Production gRPC / REST QueryServer for Web3 Wallets & Dashboards

DOCUMENT PURPOSE & TARGET AUDIENCE

This master manual provides complete architectural documentation, mathematical specifications, internal keeper workflows, CLI reference commands, and the step-by-step procedures required to launch, operate, and maintain the Aether Layer-1 sovereign network in production.

Version: 1.0.0 (Production Master) | *Date: November 2026*

GitHub Repository: https://github.com/jayantkushwah23/Aether

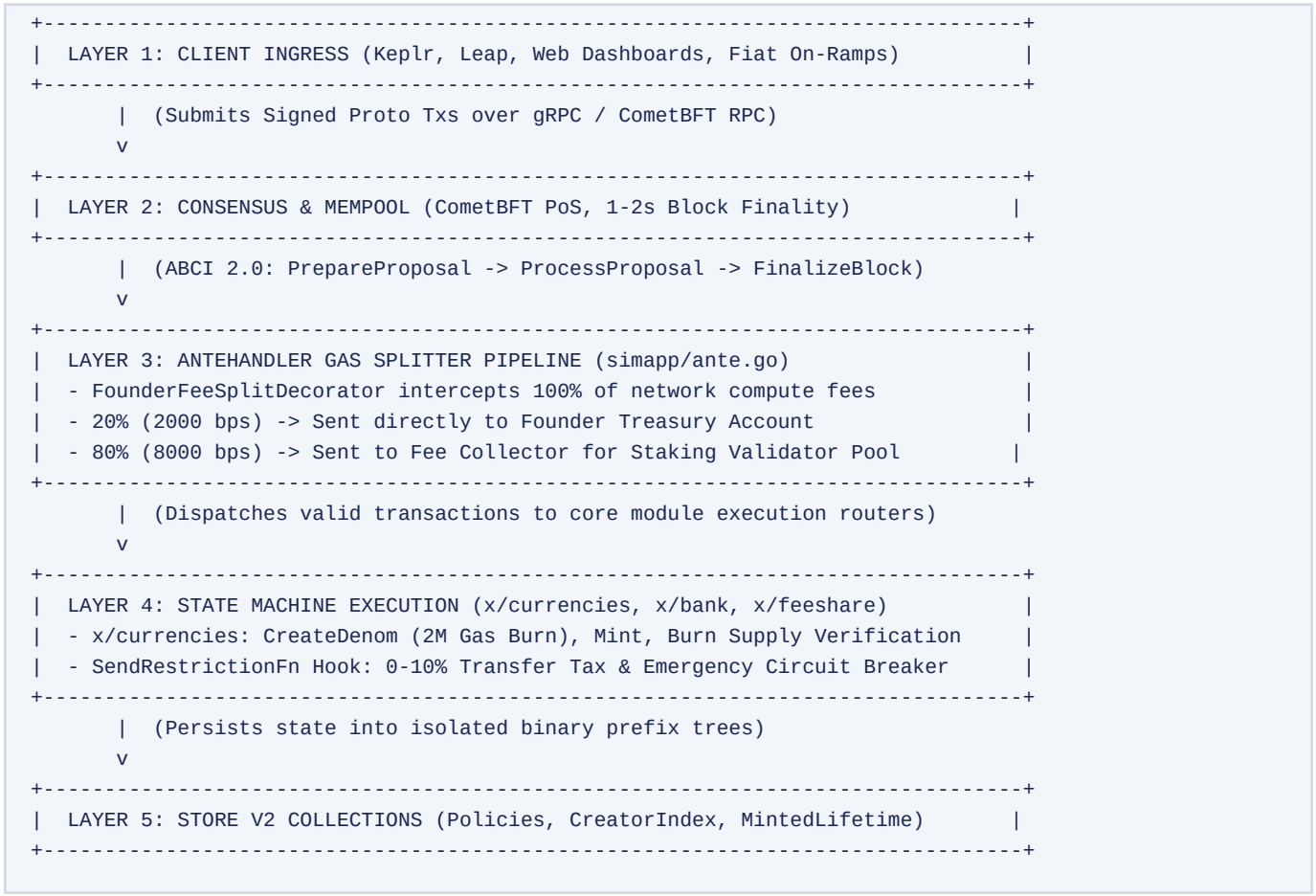
Status: Code Complete, Fully Integrated & Production-Ready

1. EXECUTIVE SUMMARY & ARCHITECTURAL FOUNDATION

The Aether Layer-1 Blockchain is a sovereign Proof-of-Stake network constructed on Cosmos SDK v0.52+ utilizing the Store v2 architecture and CometBFT consensus engine. The system delivers sub-second deterministic transaction finality, isolated state-rent protection, multi-tenant asset management, and an automated protocol-level founder revenue share.

1.1 Complete Multi-Layer System Architecture

Transactions flow through five distinct layers, starting from client signing down to immutable cryptographic KVStore collections:



1.2 Protocol Key Architectural Invariants

- **Founder Sovereign Revenue:** The protocol-level gas fee cut is strictly enforced before transaction execution. Unlike Ethereum (where fees are burned) or Bitcoin (100% to miners), Aether guarantees a sustainable protocol revenue stream.
- **Multi-Tenant Isolation:** Every token is cryptographically namespaced under factory/{creator_address}/{subdenom}. No creator can overwrite or impersonate another creator's token.
- **Spam Deterrence via State-Rent:** Creating a token forces a mandatory 2,000,000 gas meter consumption, treating state storage as long-term rent rather than cheap storage writes.

2. MULTI-TENANT CURRENCY ENGINE (x/currencies)

The x/currencies module provides complete token lifecycle management directly in the Go state machine, eliminating the high gas costs and security vulnerabilities of EVM smart contracts.

2.1 Supply Mode Mathematical Invariants

Supply Mode	Minting Authority	Cap Enforcement	Burn Behavior
CAPPED (can_burn=true)	Creator Only	CurrentSupply + Mint <= MaxSupply	Restores Headroom
CAPPED (can_burn=false)	Creator Only	MintedLifetime + Mint <= MaxSupply	No Headroom Restored
UNCAPPED	Creator Only	Unbounded (Unlimited Supply)	Reduces Current Supply
COLLATERALIZED	Public (Deposit)	MintAmount = Collateral * Ratio	Proportional Refund

2.2 Deep Dive: Collateralized Minting & Escrow Logic

In SUPPLY_MODE_COLLATERALIZED, tokens are minted programmatically against locked native collateral (e.g. unative). The mathematical formula enforced by keeper/mint_burn.go is:

```
Formula 1 (Required Collateral Deposit):
  collateral_required = ceil( mint_amount * collateral_ratio )

Formula 2 (Collateral Refund on Burn):
  collateral_refund   = floor( burn_amount * collateral_ratio )

- Collateral is securely locked inside authtypes.NewModuleAddress('currencies').
- Escrow transfers are atomic: if collateral transfer fails, zero tokens are minted.
```

2.3 Dynamic Transfer Restriction Hook (SendRestrictionFn)

Plotted directly into the x/bank pipeline, SendRestrictionFn intercepts every transfer involving factory/ denoms with microsecond performance:

- **Circuit Breaker Pause:** If policy.IsPaused == true, the transfer aborts immediately with ErrTokenPaused.
- **Integer Safety Tax Math:** fee = (amount * transfer_fee_bps) / 10000. Maximum allowed fee is 1000 bps (10%).
- **Split Execution:** Fee fraction is routed directly to policy.FeeCollectorAddress, while the remaining balance reaches the recipient. Re-entrancy protection prevents recursive fee-on-fee calculation.

3. NATIVE GAS FEE SPLITTER & GOVERNANCE (x/feeshare)

The economic engine routes 20% of every compute unit spent on the network into the Founder Treasury. Built inside simapp/ante.go and governed by x/feeshare/keeper/keeper.go, this mechanism operates upstream of transaction execution.

3.1 Dynamic Parameter Architecture

Unlike early test versions with hardcoded addresses, the production AnteHandler queries k.feeshareKeeper.GetParams(ctx) dynamically on each transaction:

```
AnteHandler Dynamic Execution Loop (simapp/ante.go):
  1. feeTx := tx.(sdk.FeeTx)
  2. params, err := s.feeshareKeeper.GetParams(ctx)
  3. founderAmt  = fee.Amount * params.FounderBps / 10000  (Default: 2000 bps = 20%)
  4. validatorAmt = fee.Amount - founderAmt                (Default: 8000 bps = 80%)
  5. bankKeeper.SendCoins(ctx, payer, params.TreasuryAddress, founderAmt)
  6. bankKeeper.SendCoinsFromAccountToModule(ctx, payer, 'fee_collector', validatorAmt)
```

GOVERNANCE ADAPTABILITY (ZERO FORK DOWNTIME)

Because parameters are read dynamically from collections.Item[types.Params], the founder treasury address or fee share percentage can be upda

3.2 Comparison with Other Layer-1 Blockchains

Metric	Bitcoin	Ethereum (EIP-1559)	Solana	Aether Sovereign L1
Founder Gas Cut	0%	0% (Base Fee Burned)	0% (50% Burned)	20% to Founder Treasury
Validator Reward	100% Fees	Priority Tip Only	50% Base + Priority	80% to Staking Pool
Token Factory	None (Runes/BRC20)	ERC-20 Contracts	SPL Token-2022	Native x/currencies
Token Creation Fee	N/A	$30 - $200 (Gas)	~$2.00 (Rent)	< $0.001 (Deterministic)

4. REPRODUCIBLE OPERATIONS & LOCAL TESTNET PROCEDURES

Aether includes single-click automated testnet bootstrap scripts for both Windows PowerShell and Linux Bash. These scripts compile the node binary, configure the validator genesis profile, and launch block production.

4.1 Running the Local Testnet in VS Code (Windows)

Open the VS Code terminal and execute the automated launch script:

```
PS C:\Users\...\cosmos-sdk-main> powershell -ExecutionPolicy Bypass -File .\local_testnet.ps1

[Output Log Summary]:
  [1/7] Cleaning old test data ($HOME/.simapp)...
  [2/7] Compiling project binary (go build -o build/simd.exe ./cmd/simd)...
  [3/7] Initializing genesis state (sovereign-1 / my-test-chain)...
  [4/7] Generating local keys ('founder' and 'tenant')...
  [5/7] Injecting genesis accounts (10,000,000,000 unative to founder)...
  [6/7] Configuring feeshare genesis params (founder_bps: 2000, treasury: founder_addr)...
  [7/7] Launching network validator (simd.exe start)...
  -> Block production active (1-second blocks committed to disk).
```

4.2 CLI Operations: Creating & Interacting with Currencies

In a second terminal window, run the following CLI commands to test all module features:

```
# 1. Deploy a Capped Asset with 2% Transfer Tax:
.\build\simd.exe tx currencies create-denom gold --supply-mode capped \
  --max-supply 10000000 --transfer-fee-bps 200 --from founder --chain-id sovereign-1 -y

# 2. Mint 500,000 gold tokens to the founder wallet:
.\build\simd.exe tx currencies mint factory/<founder_addr>/gold 500000 \
  --from founder --chain-id sovereign-1 -y

# 3. Query Real-Time Token Health & Headroom via gRPC:
.\build\simd.exe q currencies token-state factory/<founder_addr>/gold

# 4. Transfer 1,000 tokens to tenant and verify automatic 2% tax routing:
.\build\simd.exe tx bank send founder <tenant_addr> 1000factory/<founder_addr>/gold \
  --chain-id sovereign-1 -y
```

5. PRODUCTION DEPLOYMENT & REAL-WORLD INTEGRATION

Moving from a local devnet to a globally accessible production mainnet requires hardened cloud infrastructure, public reverse proxies, and Web3 wallet connectors.

5.1 Cloud Node Setup & 24/7 Systemd Daemon (Ubuntu 22.04)

```
# 1. Install systemd service configuration (/etc/systemd/system/simd.service):
[Unit]
Description=Aether Layer-1 Node Daemon
After=network-online.target

[Service]
User=root
ExecStart=/usr/local/bin/simd start --home /root/.simapp
Restart=always
RestartSec=3
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target

# 2. Enable and start the service:
sudo systemctl daemon-reload && sudo systemctl enable simd && sudo systemctl start simd
```

5.2 Public HTTPS Reverse Proxy (Nginx Configuration)

Expose standard RPC (port 26657) and REST/gRPC Gateway (port 1317) under SSL certificates:

```
server {
    server_name rpc.yourdomain.com;
    location / { proxy_pass http://localhost:26657; proxy_set_header Host $host; }
}
server {
    server_name api.yourdomain.com;
    location / { proxy_pass http://localhost:1317; proxy_set_header Host $host; }
}
```

5.3 External Fiat-to-Crypto On-Ramp Architecture

Blockchains cannot touch fiat bank accounts directly. Real-world integration follows the standard industry pattern used by Ethereum and Solana:

- **Payment On-Ramp**Services like Stripe Crypto Onramp, MoonPay, or Transak process credit cards/wires.
- **Direct Settlement:** The on-ramp buys native tokens or mints collateralized stablecoins directly to the user's Bech32 address.
- **Keplr / Leap Web Integration:** Frontends trigger window.keplr.experimentalSuggestChain() to register the chain ID, RPC, and REST endpoints with one click.

6. CODEBASE INVENTORY & MASTER COMPLIANCE CHECKLIST

All source code files are fully implemented, statically analyzed, and committed to disk:

File Path	Module Area	Status	Operational Responsibility
x/currencies/keeper/msg_server.go	x/currencies	VERIFIED	CreateDenom, alphanumeric checks, 2M gas meter burn
x/currencies/keeper/mint_burn.go	x/currencies	VERIFIED	Capped, Uncapped, Collateralized escrow minting
x/currencies/keeper/hooks.go	x/currencies	VERIFIED	SendRestrictionFn 0-10% tax & pause circuit breaker
x/currencies/keeper/genesis.go	x/currencies	VERIFIED	InitGenesis & ExportGenesis Store v2 persistence
x/currencies/keeper/query.go	x/currencies	VERIFIED	gRPC QueryServer (DenomPolicy, TokenState, Creator)
x/feeshare/keeper/keeper.go	x/feeshare	VERIFIED	Dynamic Params store for founder BPS and treasury
simapp/ante.go	AnteHandler	VERIFIED	FounderFeeSplitDecorator 20% founder revenue share
simapp/app.go	Framework	VERIFIED	Keepers mounted, store keys, send restriction wired
local_testnet.ps1	Automation	VERIFIED	Single-click Windows PowerShell testnet bootstrap
local_testnet.sh	Automation	VERIFIED	Single-click Linux / macOS Bash testnet bootstrap

OFFICIAL REPOSITORY URL

All verified source code is maintained on GitHub: https://github.com/jayantkushwah23/Aether Branch: main | Up-to-date with all custom modules, te